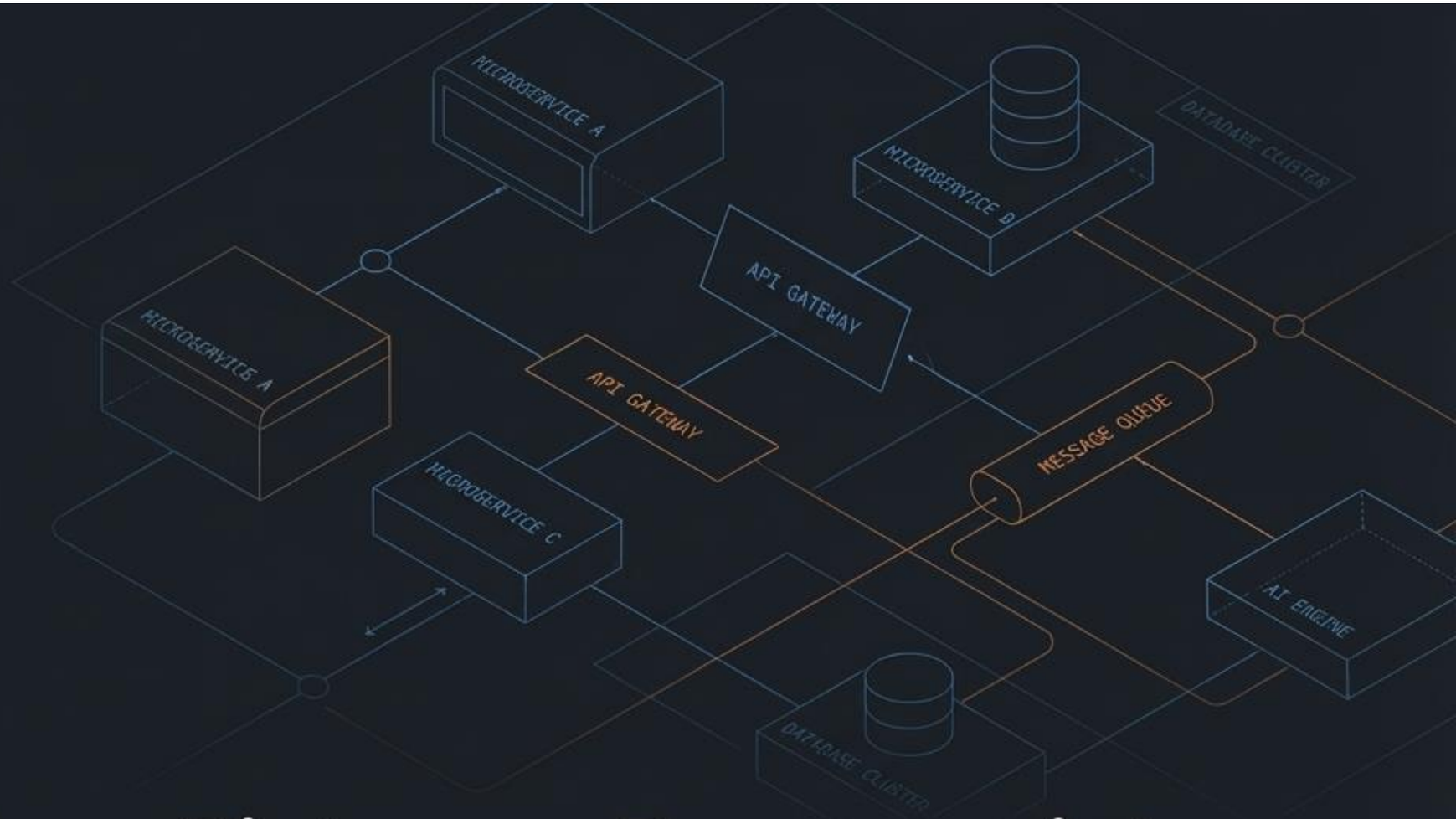
JOEDAYZ academy



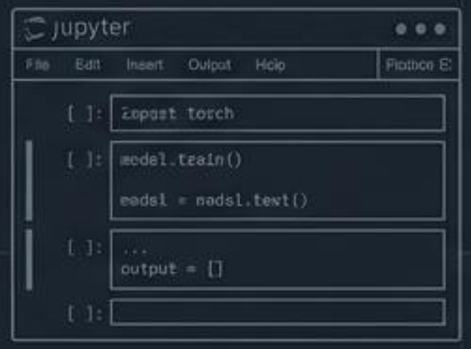
Integrando IA en Sistemas Empresariales: De Workflows a Inteligencia Agéntica

Construyendo la próxima generación de aplicaciones con Java y LangChain4j

NotebookLM

No estamos entrenando modelos; los estamos integrando en el ecosistema empresarial

Creación de Modelos
Data Science



Python, PyTorch, Entornos de Investigación.

El Mito: La IA es territorio exclusivo de Python.

Integración en Producción
AI Engineering

Java, Lógica de Negocio, APIs, Rendimiento a Escala.

La Realidad: Java ofrece robustez y un rendimiento superior (hasta 100x) para sistemas críticos.

El Cambio de Rol: Tu objetivo no es ser Data Scientist. Eres un AI Engineer encargado de orquestar LLMs con lógica de negocio.

NotebookLM

LangChain4j actúa como el adaptador universal entre Java y el ecosistema de IA

NotebookLM

Ciudadano de primera clase en el ecosistema moderno de Java

Integración Nativa: No es un script aislado; utiliza extensiones y starters oficiales.

Inyección de Dependencias: Gestión de componentes de IA como beans estándar.

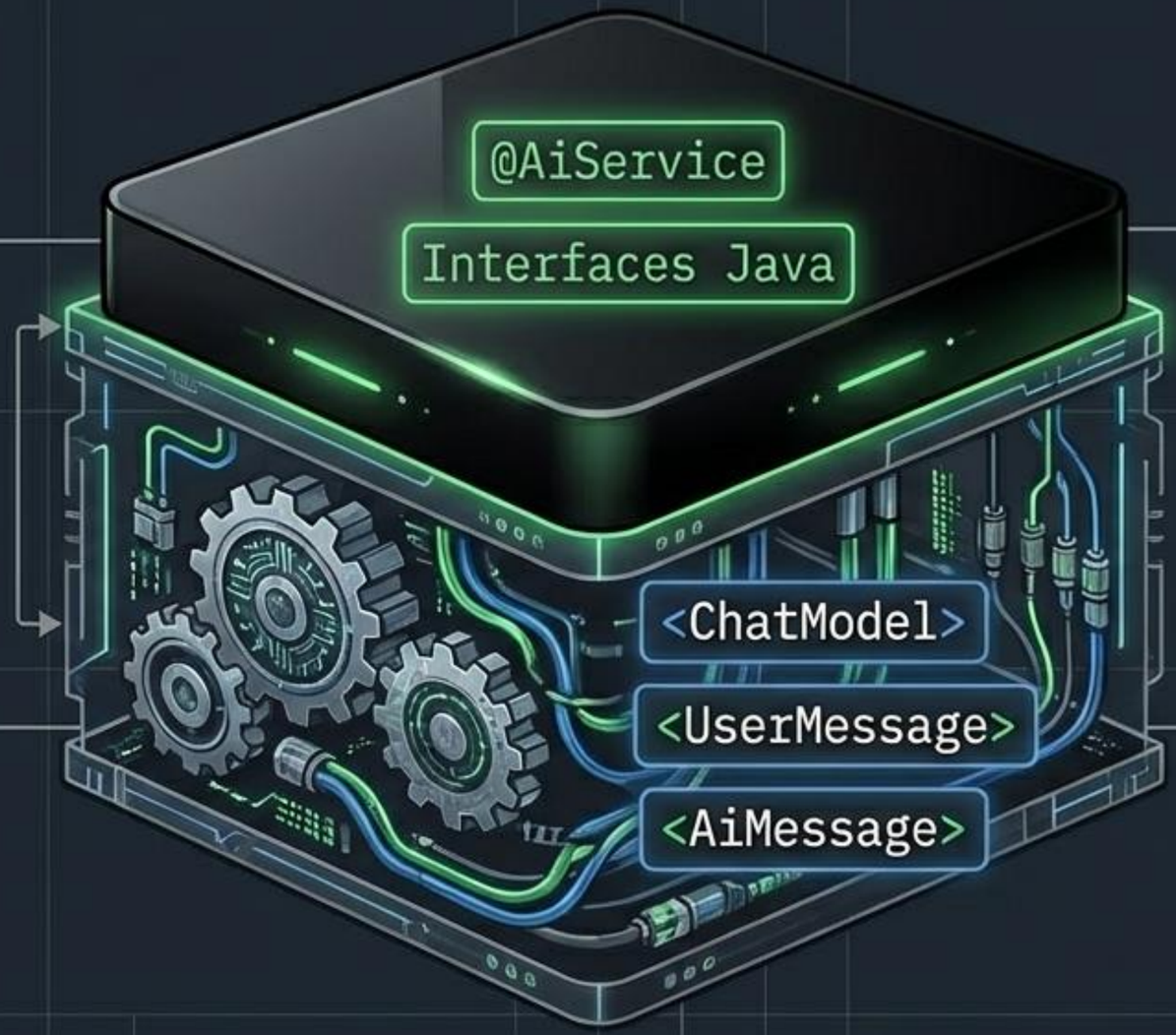
Configuración Declarativa: Gestiona múltiples modelos y RAG en propiedades del framework.

NotebookLM

Dos niveles de abstracción para equilibrar control y productividad declarativa

Alto Nivel
(AI Services)

Bajo Nivel
(Primitivas)



Oculta el boilerplate. Defines interfaces limpias y dejas que la librería inyecte dinámicamente el estado y el formato de las respuestas.

Libertad total. Interactúas directamente con tokens, construyes historiales a mano y gestionas el glue code.

NotebookLM

Los LLMs no tienen estado. La Ventana Deslizante simula la memoria.

El Reto:	La Solución:
Cada petición a un LLM es aislada y comienza desde cero.	**LangChain4j** reenvía automáticamente el historial en memoria, eliminando mensajes antiguos para respetar los límites de tokens.

NotebookLM

Function Calling: Transformando métodos de Java en habilidades del LLM

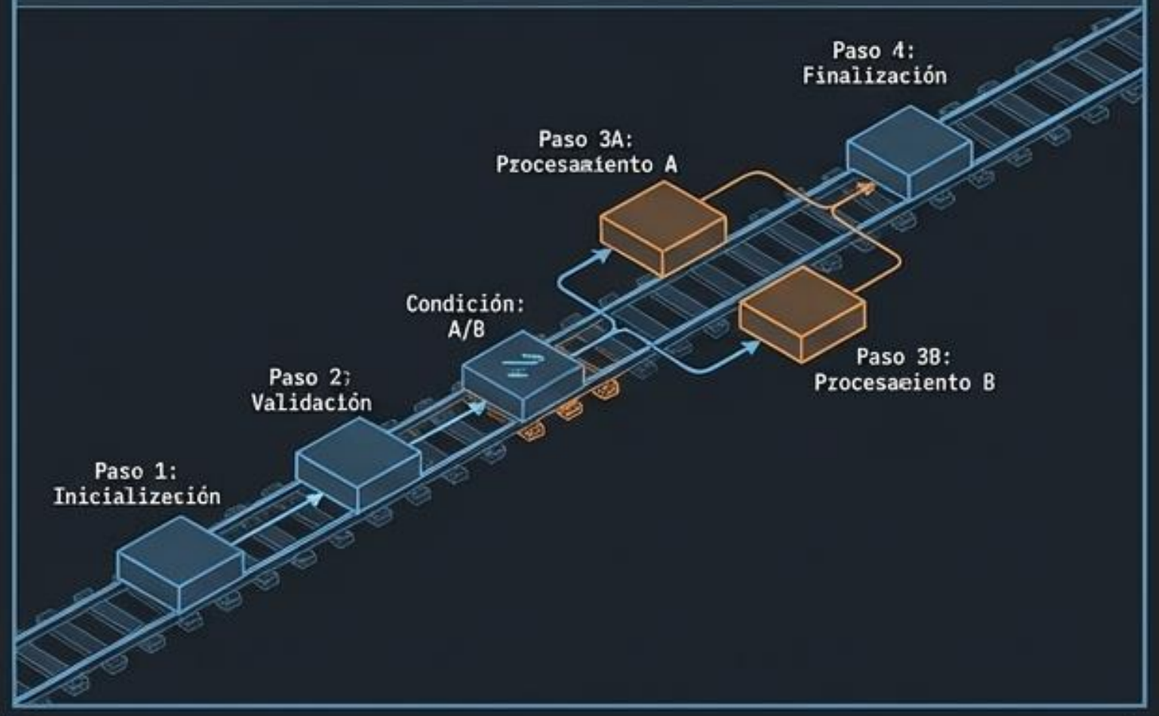
Mediante la anotación @Tool, el LLM no ejecuta el código; devuelve un JSON pidiendo a Java que lo ejecute por él.

NotebookLM

El Paradigma de Ejecución: ¿Quién tiene el control del plan?

La evolución avanza desde flujos de trabajo programáticos hacia delegación autónoma.

Camino A: Workflows



El código Java orquesta el flujo. Ejecución lineal, condicional y estrictamente predecible.

Camino B: Agentes

El LLM decide la ruta en tiempo real. Ejecución emergente, autónoma y adaptativa.

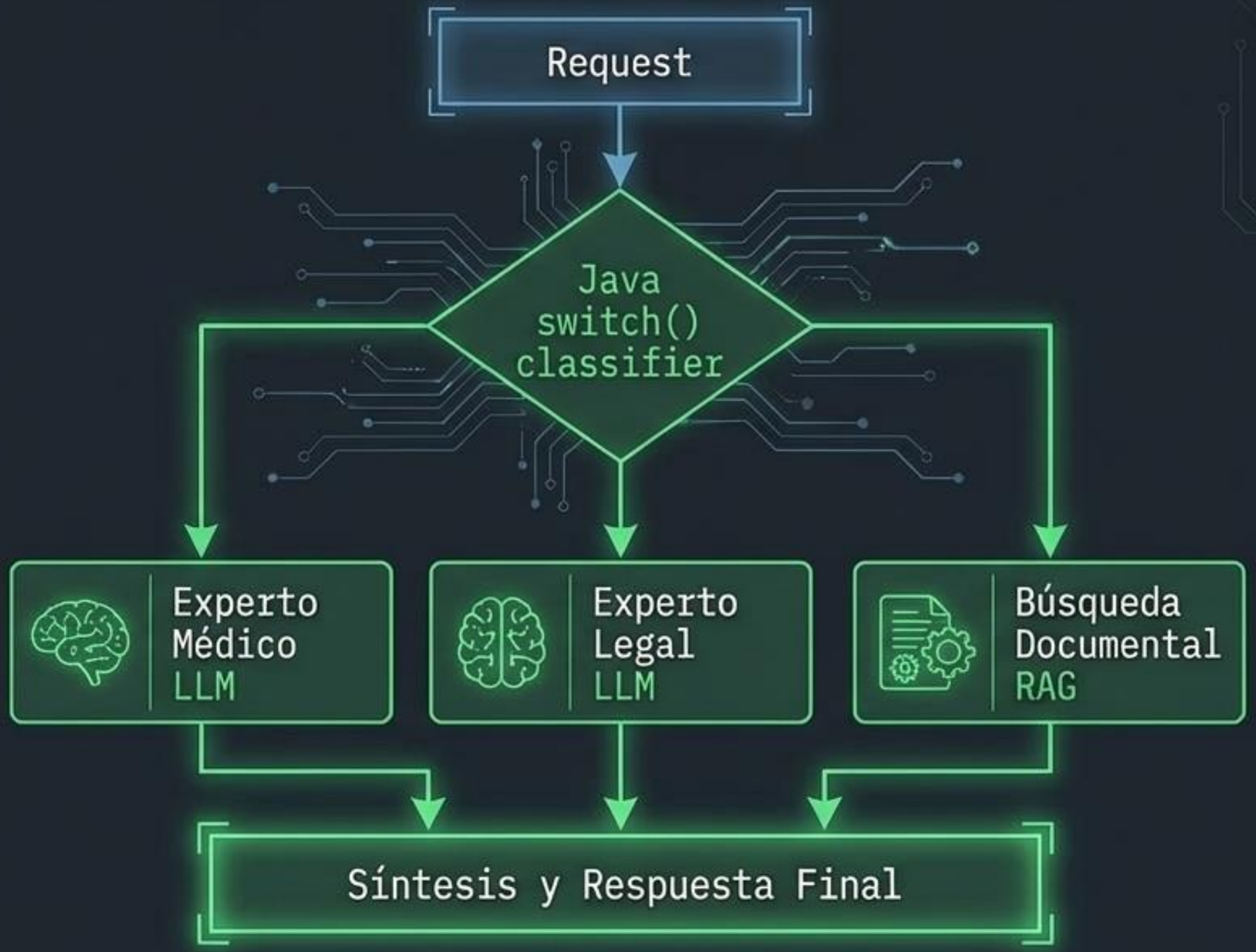
NotebookLM

Patrones de Workflow: Orquestación controlada y determinista

Características:
Orquestación desde el exterior.
El desarrollador escribe el "glue code".

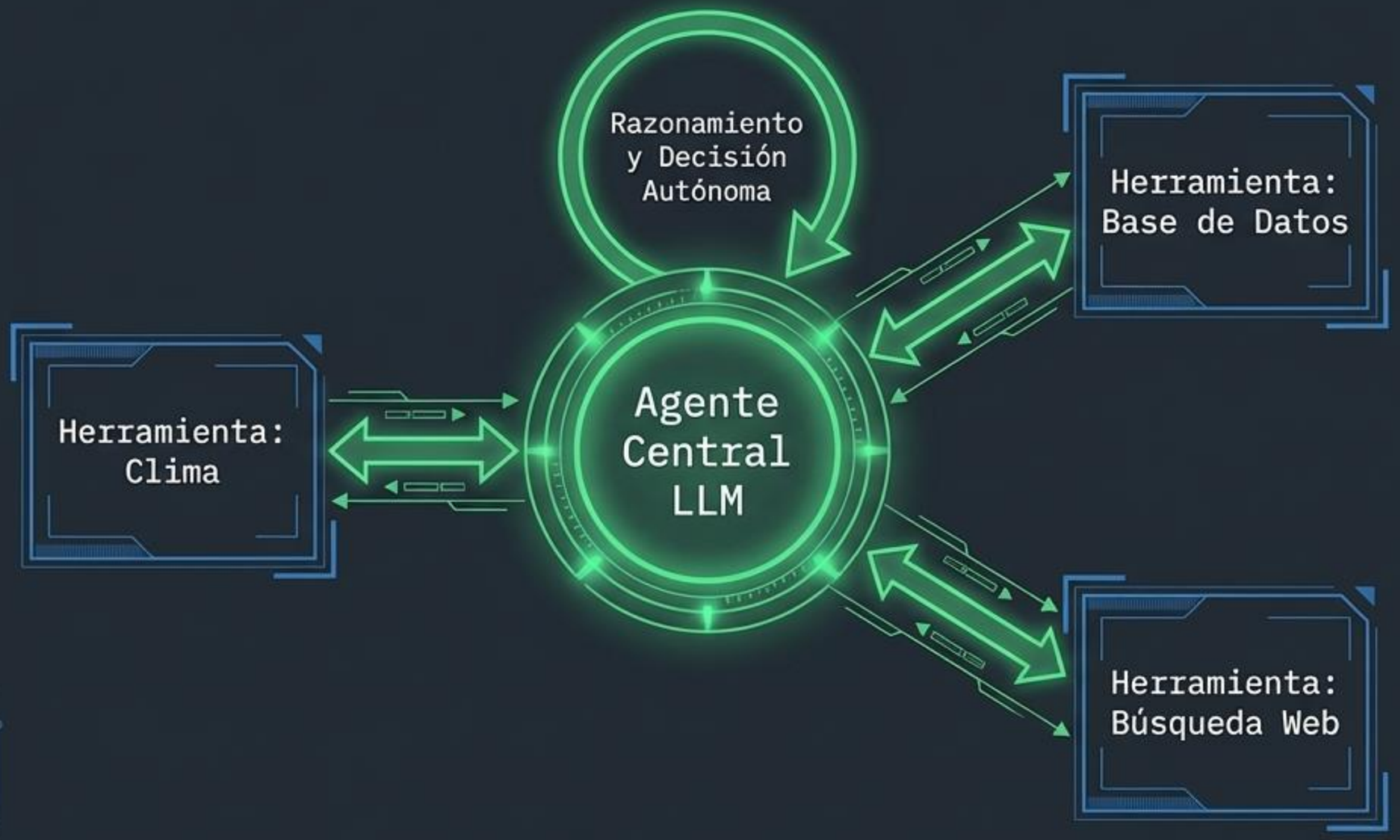
Ejemplos: Prompt chaining, Paralelización, Mixture of Experts explícito.

Ventajas: Alta predecibilidad, fácil de auditar, rastreabilidad exacta para depuración.



NotebookLM

Patrones Agénticos: Delegación total del plan de acción



Comportamiento Dinámico

Cero bloques condicionales en Java. Le das un objetivo y herramientas, y el agente decide llamar simultáneamente a múltiples expertos según lo deduzca.

El Compromiso

Flexibilidad extrema frente a imprevisibilidad. Aumenta la latencia por procesamiento de múltiples ciclos y existe riesgo de alucinación en invocación de herramientas inexistentes.

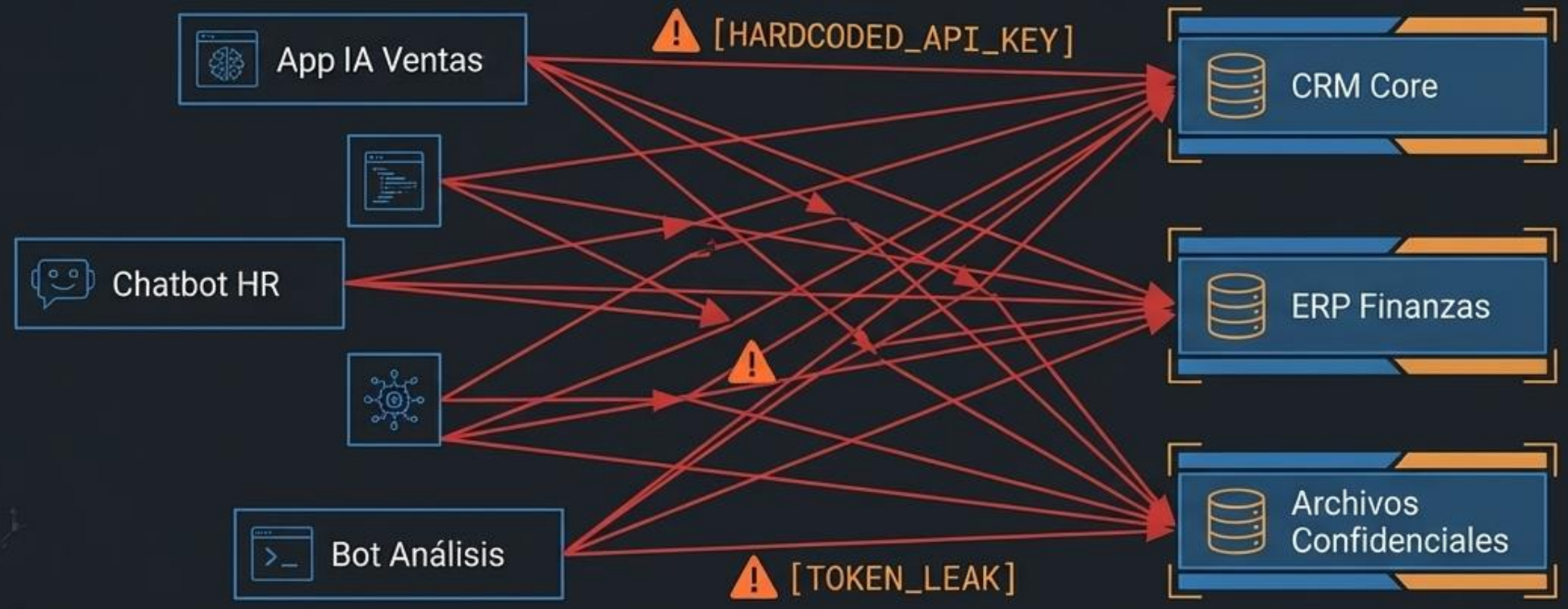
NotebookLM

Matriz de Decisión: ¿Cuándo usar Workflows vs. Agentes?

Criterio Arquitectónico	Workflow Programático	Agente Autónomo
Predecibilidad	Muy Alta (Lineal)	Baja (Comportamiento emergente)
Esfuerzo de Código (Glue Code)	Alto (Mucha lógica condicional)	Mínimo (Cero código de enrutamiento)
Flexibilidad ante imprevistos	Rígida (Falla si el caso no está programado)	Extrema (El LLM deduce la solución)
Latencia de Respuesta	Baja / Optimizada / Paralela	Alta (Múltiples ciclos de ida y vuelta)
Facilidad de Testing y Depuración	Sencilla (Tests de aislamiento clásico)	Compleja (Caja negra estocástica)

NotebookLM

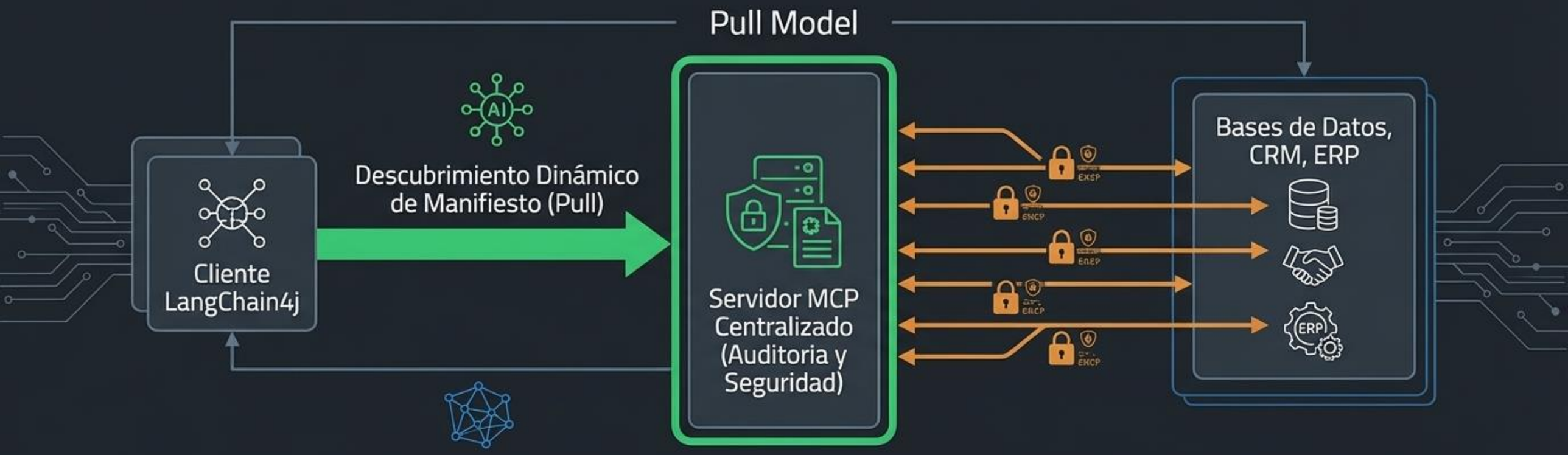
El límite del crecimiento: Shadow IT y el caos de las herramientas



El Problema de Industrializar (Push Model): A medida que escalamos, programar y compartir accesos a herramientas de forma individual expone tokens empresariales, duplicando vulnerabilidades y destruyendo la auditoría.

NotebookLM

Model Context Protocol (MCP): Industrializando el acceso a herramientas



El Nuevo Estándar	Separación de Responsabilidades
Cambia el paradigma de inyectar código duro (Push) a descubrir servidores seguros en tiempo real (Pull).	Equipos de seguridad gestionan el Servidor MCP con credenciales reales; los Agentes consumen sin conocer los secretos.

NotebookLM

Pruebas en Entornos No Deterministas: Cuando assertEquals ya no funciona

Testing Estricto (JUnit Clásico)

```
@Test
public void testRespuestaExacta() {
    String respuesta = chatbot.preguntar("¿Dónde está el gato?");
    assertEquals("El gato está en la mesa", respuesta);
}
```

```
[FAILED] Expected: "El gato está en la mesa"
 Actual: "Veo un animal con apariencia felina sobre un mueble"
```

Los LLMs son estocásticos. Las pruebas de aserción estricta generan falsos positivos de error por simples variaciones de redacción.

Evaluación Estocástica (LLM-as-a-judge)

```
@Test
public void testSimilitudSemantica() {
  String respuesta = chatbot.preguntar("¿Dónde está el gato?");
  assertThat(respuesta).isSemanticallySimilarTo("El gato está en la mesa");
}
```

```
[PASSED] Similitud del Coseno: 0.94
```

Nuevas Estrategias: Usar puntuadores matemáticos (Similitud del Coseno) o un segundo modelo (Juez LLM) para validar el cumplimiento semántico.

NotebookLM

Síntesis: Java está listo para liderar la IA Empresarial

El Stack Moderno de IA en Java

Nivel 4	**Capa de Negocio:** Workflows Estrictos & Agentes Autónomos
Nivel 3	**Capa Cognitiva:** LangChain4j (Memoria, Herramientas, AI Services)
Nivel 2	**Ecosistema:** Quarkus / Spring Boot / Servidores MCP Centralizados
Nivel 1	**Infraestructura:** Modelos Nube / SLMs Locales (Ollama) / Vector Stores

De Scripts a Sistemas:
Orquestación segura, tipificada y escalable sobre la JVM.

El Mejor de Dos Mundos:
El poder cognitivo de los LLMs con el ecosistema empresarial de Java.

El Código Estructurado Importa:
La arquitectura y el diseño de software son ahora más relevantes que nunca.

NotebookLM